

INTRODUCTION // 課題と目的

なぜMVCから移行するのか？ ヘキサゴナルアーキテクチャ完全理解ガイド

「インターフェースが多すぎる」「どこに書けばいいかわからない」という疑問を解消し、MVCの構成要素との1対1の対応関係と、設計変更のメリットを整理します。

⚠ 従来のMVCで起きていた課題

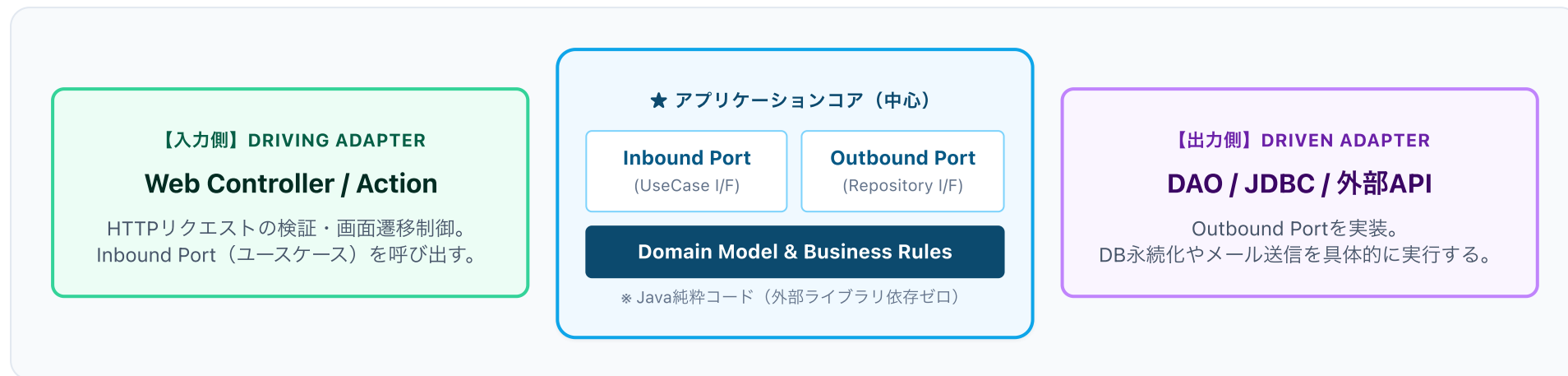
- DBスキーマ（ORM/テーブル定義）に業務ルールが引っ張られる
- Serviceクラスが肥大化し、数千行の「Fat Service」になりやすい
- 単体テストでDB接続や重いモックが必須になり、テスト実行が遅い
- メール送信や外部APIの仕様変更がコア業務に波及する

🎯 ヘキサゴナルで解決されること

- 純粋なビジネスルールがDB・フレームワークから完全に独立する
- DB接続なし・モックなしでミリ秒単位で動く軽量な単体テストが可能
- DBや外部通信は「差し替え可能なプラグイン」になる
- 各クラスの責務が単一になり、仕様変更の影響範囲が局所化される

ヘキサゴナルの核: 「中心」と「ポート&アダプター」

依存の矢印は必ず「外側から中心」に向かいます。ドメインはDBやWebの存在を知りません。



💡 **依存性の逆転 (DIP):** ドメインはDB (DAO) を呼びたいが直接依存しない。「保存できる口 (Outbound Port)」だけをコア側で定義し、外側のDAOがそれを実装します。

「MVCのこれ、ヘキサゴナルでどこ？」完全対応マップ

既存プロジェクトの「Action / Service / DAO」がどうマッピングされるかの一覧です。

従来のMVC要素	ヘキサゴナルでの呼び名	具体的な責務	配置パッケージ例
Action / Controller	Driving Adapter (Web)	HTTPリクエスト取得、パラメータ検証、UseCase呼出、画面遷移	adapter.in.web
Serviceのインターフェース	Inbound Port	外部から実行可能なユースケースの入力規約（メソッド定義）	application.port.in
Serviceの実装クラス	UseCase Service	業務処理フローの進行役。ドメインモデルを操作してOutbound Portを呼ぶ	application.service
DAOのインターフェース	Outbound Port	コア層が永続化や外部通知に求める能力の宣言（依存の逆転）	application.port.out
DAO実装 / JDBC / SQL	Driven Adapter (DB)	Outbound Portを実装し、SQL発行・DBトランザクションを実行	adapter.out.persistence

社員登録機能のコード比較 (Before / After)

ServiceがDBや外部に直結していた構造から、インターフェース経由への変化を確認してください。

✗ 【Before】 従来のMVC (ServiceがDAOに直接依存)

```
// ServiceがDAOの具体クラスに依存
public class ShainService {
    private ShainDAO shainDao = new ShainDAO();

    public void registShain(ShainForm form) {
        // 業務ルールがService内に散乱
        if (form.getName() == null) {
            throw new IllegalArgumentException("必須");
        }

        ShainDTO dto = new ShainDTO(form.getName());
        shainDao.insert(dto); // DB直結
    }
}

/* 課題: 単体テスト時にDBが起動していないと
ShainServiceのテストが動かない */
```

○ 【After】 ヘキサゴナル (Port経由・依存逆転)

```
// 1. コア層: ポート(Interface)にのみ依存
public class RegistShainService
    implements RegistShainUseCase {

    private final ShainRepositoryPort repo;

    public RegistShainService(ShainRepositoryPort r) {
        this.repo = r;
    }

    @Override
    public void execute(RegistShainCommand cmd) {
        Shain shain = new Shain(cmd.getName());
        repo.save(shain); // SQLの存在を知らない
    }
}

/* 利点: テスト時は repo にインメモリの
ダミークラスを渡すだけで1msでテスト完了 */
```

ディレクトリ探索ガイド: 「修正時はどこを見る？」

目的ごとに開くべきディレクトリが完全に分離されています。

domain/

見るべきタイミング:

「計算ロジックが変わった」「入力値の妥当性ルールが変わった」とき。

- Shain.java
- ShainName.java
- ValidationException.java

application/

見るべきタイミング:

「社員登録の後にメールを送る手順を追加したい」など処理フローが変わったとき。

- port/in/ (UseCase I/F)
- port/out/ (Repo I/F)
- service/ (業務フロー)

adapter/

見るべきタイミング:

「JSPのパラメータ名が変わった」「SQLのクエリやテーブル定義を変えたい」とき。

- in/web/ (Action/Servlet)
- out/persistence/ (JDBC/DAO)
- out/mail/ (SMTP通知)

よく出る疑問と回答（FAQ）

勉強会でメンバーから必ず出る質問と、その納得感ある回答です。

Q1. クラスやインターフェースが増えて面倒ではないですか？

A. 一見ファイルは増えますが、1ファイルあたりのコード行数は激減します。変更時に「この修正がどこに影響するか」を探す時間が大幅に短縮され、結果的に開発効率が上がります。

Q2. ドメインのEntityとDAOのDTOで同じようなクラスがあるのはなぜ？

A. ドメインは「業務のルール」、DTOは「DBテーブルの都合」です。目的が異なるため、分離することで「DBカラムの都合（VARCHAR長さやテーブル結合）が業務ルールに悪影響を与える」のを防ぎます。

Q3. 実装するときにはどの順序で書けばいい？

A. **Domain Model → Inbound Port (UseCase) → Application Service → Adapter (Web / DAO)** の順です。まず業務ロジックをテスト駆動で完成させてから、最後に外側のDBや画面を繋ぎます。